

VIXOR - PLAN PHASE 2

خطة تنفيذية مفصلة تحسينات VIXOR الجوهرية

ربط محرك التحليل بالواجهة، دورة حياة الاشارات، محرك المشاركة،
مراقبة الاسعار الحية - مع مرجع QuantDinger المفتوح المصدر

الملخص التنفيذي

يتضمن هذا التقرير خطة تنفيذية مفصلة لتطوير اربع ميزات جوهرية في تطبيق VIXOR، وهو رفيق الصفقة (Trade Companion) الذي يعمل كتطبيق ويب داخل تليجرام. التطبيق مبني على TanStack Start مع React 19 و Supabase و Vite وموزع على Vercel. يحتوي المشروع حاليا على 363 ملف TypeScript و 16 نطاق عمل (DDD) و 92 مكتبة وقت التشغيل و 33+ صفحة واجهة مستخدم تعمل بالكامل.

بناء على تحليل شامل للمشروعين VIXOR الحالي ومشروع QuantDinger المفتوح المصدر، تبين ان VIXOR يمتلك بالفعل معظم البنية التحتية المطلوبة. المحرك التحليلي (SMC/ICT) يعمل بالكامل بنظام حتمي بدون مفاتيح API خارجية. نظام الاشارات يتم توليده بشكل دوري ويخزن في قاعدة البيانات. اتصال Binance WebSocket موجود ويعمل للاسعار الحية. نظام الاشعارات يدعم تليجرام والبريد والتطبيق نفسه.

النتيجة الرئيسية: لا نحتاج لبناء أنظمة من الصفر. نحتاج لربط الأنظمة الموجودة ببعضها وتحسين نقاط الضعف المحددة. `serve` ي `QuantDinger` كمرجع تصميمي وليس كمصدر كود مباشر لأنه مكتوب بـ `Python` و `VIXOR` يعمل بـ `TypeScript` في بيئة `Serverless`.

الميزات الأربع المطلوبة وتقييم حالتها الحالية هي كالتالي: أولا، ربط محرك التحليل بالواجهة وهو يعمل فعلا لكن زر المشاركة في صفحة النتائج وهمي بدون وظيفة. ثانيا، دورة حياة الاشارات من `pending` الى `tp_hit/sl_hit` وهذا غير موجود تماما. ثالثا، محرك المشاركة على `X` و `Telegram` وهذا غير موجود. رابعا، مراقبة الاسعار الحية والفحص الحالي يعتمد على `Cron` وليس لحظيا.

الفصل الثاني

الحالة الحالية - ما يعمل وما لا يعمل

بعد مراجعة كل ملف في المشروع (استخدام ادوات فحص تلقائية + قراءة يدوية للملفات الحرجة)، الحالة الحالية كالتالي. هذا الفصل يركز فقط على الأنظمة المرتبطة بالميزات الأربع المطلوبة وليس قائمة شاملة لكل شيء في المشروع.

محرك التحليل المحلي (SMC/ICT) - يعمل

الملف الرئيسي: `src/domains/analysis/engine/engine.ts` (1115 سطر). خط الانابيب: بنية السوق ← مفاهيم SMC (Order Blocks, FVGs, مناطق سيولة, BOS/CHOCH) ← انماط الشموع (74 نمط) ← تشكيلات يابانية (20 تشكيل) ← انماط هارمونيك (8 انماط) ← مؤشرات فنية (RSI, MACD, BB, EMA, ADX) ← حساب المخاطرة والعرض. النظام حتمي بالكامل ولا يحتاج اي مفاتيح خارجية. API

نظام الاشارات (daily_signals) - جزئي

جدول `daily_signals` في Supabase يخزن: `pair, timeframe, recommendation, confidence, entry, stop_loss, take_profit[], reasons[], pattern`. يتم التوليد عبر `Cron` في `server/api/generate-signals.ts`. المشكلة: لا يوجد حقل `status` ولا دورة حياة (lifecycle). الإشارة تُولد وتُنسى - لا يتم تتبعها حتى تصل لل `TP` او `SL`.

زر المشاركة - معطل

في صفحة `src/routes/_authenticated/analysis.$id.tsx` السطر 1956، زر المشاركة (Share2) موجود كايقونة فقط بدون اي معالج `onClick`. الكود لا يحتوي على `navigator.share` ولا اي منطق مشاركة حقيقي. الزر وهمي تماما.

اتصال Binance WebSocket - يعمل

`src/shared/market-data/binance-ws.ts` (276 سطر) و

`src/shared/market-data/use-live-prices.ts` (183 سطر). ال Singleton BinanceWS

يعمل على جانب العميل (client-side) ويحصل على اسعار لحظية بدون API key. يُستخدم في ال Dashboard لكنه غير موصل بنظام الاشارات لمراقبة مستويات TP/SL.

فاحص التنبيهات (Alert Checker) - جزئي

`src/domains/trading/server/alert-checker.ts` (237 سطر). يعمل عبر Cron (ليس

لحظيا). يفحص شروط above/below/crosses_up/crosses_down ويرسل اشعارات. يعمل بشكل صحيح لكنه بطيء لان ال Cron يعمل كل دقائق وليس كل ثانية.

نظام الاشعارات - يعمل

`src/shared/notifications/` يحتوي على قنوات: telegram.ts, email.ts, in-app.ts,

webhook.ts. الاشعارات تعمل فعلا عبر ال notificationRouter. عند بناء نظام Signal Lifecycle, يمكن استخدامه مباشرة لارسال اشعارات TP hit / SL hit.

الفصل الثالث

تحليل مشروع QuantDinger المفتوح المصدر

مشروع QuantDinger (github.com/crazymonkey-1470/QuantDinger) هو منصة تحليل كمي كاملة مكتوبة بـ Python/Flask مع PostgreSQL و Redis. يغطي اسواق العملات المشفرة والاسهم الامريكية والصينية والفوركس والسلع عبر 12+ بورصة. هذا الفصل يحدد ما يمكن الاستفادة منه فعلا في VIXOR وما لا ينفع.

ملاحظة مهمة: QuantDinger مكتوب بـ Python (Flask + pandas + numpy). VIXOR مكتوب بـ TypeScript (TanStack Start + React) ويعمل على Vercel Serverless. لا يمكن نسخ كود Python مباشرة. ما نستفيد منه هو المفاهيم والتصميم وليس الكود نفسه.

ما ينفع من QuantDinger (مفاهيم وتصميم فقط): اولاً، صيغة الـ prompt المهيكلية للتحليل بالذكاء الاصطناعي - نمط JSON المُخرج يحتوي على trendOutlook, entrySuggestion, exitSuggestion, supportResistance, ثانياً، مفهوم Signal Lifecycle - الاشارات تمر بحالات محددة وليست مجرد نقطة بيانات. ثالثاً، نمط Data Source Factory - كل مصدر بيانات يتبع واجهة موحدة مع نظام fallback متعدد المستويات. رابعاً، انماط المؤشرات الفنية الاضافية مثل Pivot Points و Volume Ratio و Range Position.

ما لا ينفع: مكتبة المؤشرات الفنية (pandas/numpy) لا يمكن نقلها مباشرة لكن VIXOR لديه بالفعل RSI, MACD, BB, EMA, ADX, ATR وهي المؤشرات الاساسية. نظام الـ Backtest في QuantDinger اكثر تطوراً (trailing stop, multi-timeframe) لكن VIXOR لديه محرك Backtest يعمل بالفعل. نظام التداول المباشر (Live Trading) على 15 بورصة لا ينفع لان VIXOR رفيق تحليل وليس منصة تداول.

الميزة الاولى

دورة حياة الاشارات (Signal Lifecycle)

هذه هي الميزة الالهة. حالياً جدول daily_signals يخزن الاشارات كنقاط بيانات ساكنة بدون اي تتبع. الهدف هو تحويل كل اشارة الى كيان حي يُراقب من لحظة الدخول حتى الوصول للهدف او وقف الخسارة.

التصميم المستوحى من QuantDinger: في QuantDinger، كل اشارة تمر بدورة حياة واضحة. يوجد جدول pending_orders تُكتب فيه الاشارات الجديدة، ثم PendingOrderWorker يفحصها ويُنفذها. عند التنفيذ، يتحول حالتها. هذا النمط هو ما سنطبقه في VIXOR ولكن بلغة TypeScript ومتوافق مع Serverless.

الخطوة 1: انشاء جدول signal_tracking في Supabase

```

CREATE TYPE signal_status AS ENUM (
  'pending',      -- تم توليد الاشارة، لم تُراقب بعد
  'active',       -- يُراقب السعر حاليا
  'tp1_hit',      -- الهدف الاول تحقق
  'tp2_hit',      -- الهدف الثاني تحقق (كل الاهداف)
  'sl_hit',       -- وقف الخسارة تحقق
  'expired',      -- انتهت صلاحية الاشارة
  'cancelled'     -- الغاها المستخدم
);

CREATE TABLE signal_tracking (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  signal_id UUID REFERENCES daily_signals(id),
  source_type TEXT NOT NULL DEFAULT 'daily_signal',
  -- بيانات الاشارة
  pair TEXT NOT NULL,
  direction TEXT NOT NULL, -- 'BUY' or 'SELL'
  entry_price DOUBLE PRECISION,
  stop_loss DOUBLE PRECISION,
  take_profit JSONB,      -- [tp1, tp2, ...]
  -- حالة التتبع
  status signal_status NOT NULL DEFAULT 'pending',
  current_price DOUBLE PRECISION,
  previous_price DOUBLE PRECISION,
  max_favorable DOUBLE PRECISION DEFAULT 0,
  max_adverse DOUBLE PRECISION DEFAULT 0,
  hit_tp INTEGER,         -- كم هدف تحقق (0, 1, 2)
  -- توقيت
  activated_at TIMESTAMPTZ,
  resolved_at TIMESTAMPTZ,
  expires_at TIMESTAMPTZ, -- (مثلا 24 ساعة)
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- فهرس للاستعلام السريع
CREATE INDEX idx_signal_tracking_user_status
  ON signal_tracking(user_id, status);
CREATE INDEX idx_signal_tracking_active
  ON signal_tracking(status) WHERE status IN ('active', 'pending');

```

الخطوة 2: انشاء Domain جديد signal-tracking

ملف جديد: `src/domains/signal-tracking/types.ts` - يعرف واجهات TypeScript مطابقة للجدول.

ملف جديد: `src/domains/signal-tracking/functions.ts` - يحتوي على:

- `createTracking` (ينشئ سجل تتبع من اشارة), `activateTracking` (يُفعّل المراقبة),
- `resolveTracking` (يُحدّث الحالة عند TP/SL), `listActiveTrackings` (يُرجع الاشارات النشطة للمستخدم), `cancelTracking`.

ملف جديد: `src/domains/signal-tracking/monitor.ts` - يربط بين `useLivePrices` (ال WebSocket الموجود) و `signal_tracking`. يُشترك في اسعار ازواج محددة ويقارن ب TP/SL. عند الضغط: يُحدّث حالة الاشارة ويُرسل اشعار عبر `notificationRouter` الموجود.

الخطوة 3: ربط محرك المراقبة بالواجهة

انشاء هوك React جديد: `src/shared/hooks/use-signal-monitor.ts` . هذا الهوك يأخذ قائمة ازواج ويدخلها في `useLivePrices` الموجود. عند كل تحديث سعر، يفحص هل ضغط TP او SL لاي اشارة نشطة. هذه الطريقة تستفيد من البنية التحتية الموجودة (BinanceWS) بدلا بناء شيء جديد.

الملفات المتأثرة:

الملف	نوع التغيير	الوصف
<code>src/domains/signal-tracking/</code>	جديد	Domain جديد كامل (types, functions, monitor)
<code>src/shared/hooks/use-signal-monitor.ts</code>	جديد	هوك React يربط WebSocket ب Signal Tracking
<code>src/shared/supabase/types.ts</code>	تعديل	اضافة انواع <code>signal_tracking</code> الجديدة
<code>src/routes/_authenticated/signals.tsx</code>	تعديل	عرض حالة الاشارة (pending/active/tp_hit/sl_hit)
<code>src/routes/_authenticated/analysis.\$id.tsx</code>	تعديل	اضافة زر "تابع الاشارة" ينشئ <code>signal_tracking</code>
<code>src/shared/data/index.ts</code>	تعديل	اضافة <code>server functions</code> لـ Signal Tracking

الميزة الثانية

محرك المشاركة (X + Telegram)

حاليا زر المشاركة في صفحة النتائج (`analysis/$id`) هو مجرد ايقونة بدون وظيفة. الهدف هو تحويله الى محرك مشاركة حقيقي يُنتج نصا منسقا يتضمن: الزوج، التوصية، نسبة الثقة، نقطة الدخول، وقف الخسارة، الهدف، الاسباب، ورابط VIXOR للتسويق الفيروسي.

التصميم المستوحى من **QuantDinger**: في **QuantDinger** لا يوجد نظام مشاركة مباشر، لكن لديه مفهوم **Community Publishing** حيث يمكن نشر المؤشرات والاستراتيجيات

للجمهور. في VIXOR، كل تحليل يحتوي على بيانات غنية جاهزة (recommendation,) (confidence, entry, SL, TP, reasons) - كل ما نحتاجه هو تنسيقها في رسالة قابلة للمشاركة.

الخطوة 1: انشاء خدمة تنسيق المشاركة

ملف جديد: `src/shared/share/format-signal.ts` - يأخذ بيانات التحليل (من جدول analyses او daily_signals) ويُنتج نصا منسقا لكل منصة:

ل (Twitter) X: نص مضغوط (280 حرف) يحتوي على: ايموجي التوصية، الزوج، الدخول/SL/TP، نسبة الثقة، وهاشتاقات. مثال: "BTC/USDT LONG - Entry: 105,200 - SL: "103,800 - TP: 108,500 - Confidence: 78% - #VIXOR #Trading

ل Telegram: رسالة HTML اطول تحتوي على: التوصية مع لون، جدول نقاط الدخول/الخروج، الاسباب كقائمة، نسبة الثقة كشريط، رابط VIXOR في النهاية. تليجرام يدعم HTML في الرسائل لذلك يمكن تنسيقها بشكل احترافي.

الخطوة 2: ربط زر المشاركة بالوظيفة

تعديل `src/routes/_authenticated/analysis.$id.tsx` : اضافة onClick handler للزر الوهمي الحالي. عند الضغط، يعرض قائمة خيارات (X, Telegram, نسخ). ل X: يفتح نافذة `intent://twitter.com/intent/tweet` بالمعلومات. ل Telegram: يفتح نافذة `t.me/share/url` بالرسالة. ل النسخ: `navigator.clipboard.writeText`.

الخطوة 3: اضافة زر المشاركة في صفحة الاشارات

تعديل `src/routes/_authenticated/signals.tsx` : كل كارت اشارة يحصل على زر مشاركة صغير. نفس المنطق لكن البيانات تأتي من daily_signals بدلا من analyses.

الملفات المتأثرة:

الملف	نوع التغيير	الوصف
<code>src/shared/share/format-signal.ts</code>	جديد	تنسيق نص المشاركة ل X و Telegram
<code>src/shared/share/x-share.ts</code>	جديد	بناء رابط intent لتويتر
<code>src/shared/share/telegram-share.ts</code>	جديد	بناء رابط share لتليجرام
<code>src/routes/_authenticated/analysis.\$id.tsx</code>	تعديل	تفعيل زر المشاركة (السطر 1942-1957)
<code>src/routes/_authenticated/signals.tsx</code>	تعديل	اضافة زر مشاركة لكل كارت اشارة

مراقبة الاسعار الحية (Real-Time Price Monitor)

هذه الميزة مرتبطة ارتباطا وثيقا بدورة حياة الاشارات. مراقبة الاسعار تعني: عندما ينشئ المستخدم اشارة او يتابع تحليلا، يبدأ التطبيق في مراقبة سعر الزوج بشكل لحظي عبر WebSocket، وعندما يصل السعر لوقف الخسارة او الهدف يُرسل اشعار فوري.

الحالة الحالية: VIXOR لديه بالفعل كل ما يلزم لمراقبة الاسعار. BinanceWS (Singleton يربط بـ Binance useLivePrices (بدون API key) WebSocket العام هذا يعمل فعلا في الـ Dashboard لعرض اسعار العملات.

ما ينقص: وصل useLivePrices بنظام Signal Tracking بحيث يتم فحص كل تحديث سعر مقابل TP/SL الاشارات النشطة. هذا لا يحتاج اي بنية تحتية جديدة - فقط ربط المكونات الموجودة.

التصميم المستوحى من QuantDinger: في QuantDinger، المراقبة تعتمد على الـ Polling كل 5 دقائق (decide_interval). هذا بطيء جدا لتطبيق تليجرام حيث المستخدم يتوقع اشعارات فورية. في VIXOR سنستخدم WebSocket (الذي يعمل فعلا) للحصول على تحديثات لحظية ثم نُقارنها بـ TP/SL محليا على جهاز المستخدم.

الخطوة 1: تعزيز useLivePrices

تعديل `src/shared/market-data/use-live-prices.ts` : اضافة معامل callback `onPriceUpdate` اختياري يُستدعى مع كل تحديث سعر. هذا يسمح لاي مكون بالاستماع لتحديثات اسعار محددة بدون polling اضافي.

الخطوة 2: بناء useSignalMonitor hook

ملف جديد: `src/shared/hooks/use-signal-monitor.ts` . هذا الهوك: يجلب الاشارات النشطة من `signal_tracking`، يستخرج ازواجها، يُشترك في اسعارها عبر `useLivePrices`، وعند كل تحديث يفحص هل السعر ضغط TP او SL. عند الضغط: يُحدّث الحالة في Supabase ويُرسل اشعار.

الخطوة 3: تفعيل المراقبة في صفحة الاشارات

تعديل `src/routes/_authenticated/signals.tsx` : استدعاء `useSignalMonitor` في الصفحة. عند الضغط: عرض مؤشر مرئي (نقطة خضراء/حمراء) على كارت الاشارة. يمكن ايضا اضافة عمود "الحالة" في جدول الاشارات يعرض `pending/active/tp_hit/sl_hit`.

الملف	نوع التغيير	الوصف
<code>src/shared/hooks/use-signal-monitor.ts</code>	جديد	هوك يربط WebSocket بـ Signal Tracking
<code>src/shared/market-data/use-live-prices.ts</code>	تعديل	إضافة onPriceUpdate callback
<code>src/routes/_authenticated/signals.tsx</code>	تعديل	عرض حالة الإشارة + مؤشر مرئي
<code>src/routes/_authenticated/index.tsx</code>	تعديل	تفعيل المراقبة في الـ Dashboard

الميزة الرابعة

ربط محرك التحليل بالواجهة

محرك التحليل (Analysis Engine) متصل فعلا بالواجهة عبر صفحة `analyze/` (رفع صورة ← VLM ← محرك SMC/ICT ← عرض النتائج) و صفحة `analysis/$id/` (عرض النتائج التفصيلية). لكن هناك فجوات محددة يجب ملؤها.

الفجوة 1: زر المشاركة وهمي - تم تغطيتها في الميزة الثانية (Share Engine).

الفجوة 2: لا يمكن تحليل زوج بدون رفع صورة - حاليا المستخدم يجب ان يرفع صورة شارت. لكن محرك التحليل يدعم بالفعل التحليل المباشر عبر الزوج عبر دالة `quickAnalyze` في `src/domains/analysis/functions.ts`. هذه الدالة تأخذ `pair` و `timeframe` و `tradingStyle` وتجلب بيانات حقيقية من `Binance/ TwelveData` ثم تُشغل المحرك. كل ما ينقص هو زر "تحليل سريع" لا يتطلب صورة.

الخطوة 1: اضافة زر تحليل سريع في صفحة الاشارات

تعديل `src/routes/_authenticated/signals.tsx` : اضافة زر "تحليل الان" بجانب كل اشارة (زوج + timeframe معروفين). هذا الزر يستدعي `quickAnalyze` ويعرض النتيجة مباشرة او ينتقل لصفحة التحليل التفصيلية.

الخطوة 2: اضافة تحليل سريع في الـ Dashboard

تعديل `src/routes/_authenticated/index.tsx` : في قسم Quick Actions، اضافة خيار "تحليل سريع" يفتح قائمة بالازواج الشائعة. عند الاختيار، يُشغل `quickAnalyze` ويعرض النتيجة.

الخطوة 3: ربط التحليل بدورة حياة الإشارة

بعد اي تحليل (سواء من رفع صورة او تحليل سريع)، اضافة زر "تابع هذه الاشارة" ينشئ سجل في signal_tracking ويُفَعّل المراقبة اللحظية.

الملفات المتأثرة:

الملف	نوع التغيير	الوصف
src/routes/_authenticated/signals.tsx	تعديل	اضافة زر "تحليل الان" لكل اشارة
src/routes/_authenticated/index.tsx	تعديل	اضافة تحليل سريع في Quick Actions
src/routes/_authenticated/analysis.\$id.tsx	تعديل	اضافة زر "تابع الاشارة" ينشئ signal_tracking
src/components/vixor/atoms.tsx	تعديل	اضافة مكونات مرئية لحالة الاشارة

الفصل الثامن

خطة التنفيذ والترتيب

الترتيب مهم. كل ميزة تعتمد على سابقتها. لا يمكن بناء Share Engine قبل ان يكون هناك تحليل حقيقي لShareه. ولا يمكن بناء Price Monitor قبل ان تكون هناك اشارات نشطة لمراقبتها.

المخاطرة	الملفات المعدلة	الملفات الجديدة	المهام	المرحلة
منخفضة إضافة فقط، لا تعديل كود موجود	supabase/types.ts	signal-tracking/types.ts signal-tracking/functions.ts supabase migration SQL	إنشاء جدول signal_tracking في Supabase تحديث types.ts إنشاء signal-tracking domain	المرحلة 1 الاساسيات
منخفضة إضافة فقط، لا تعديل منطق موجود	analysis/\$id.tsx signals.tsx	shared/share/format-signal.ts shared/share/x-share.ts shared/share/telegram-share.ts	بناء format-signal.ts signal.ts بناء x-share.ts و telegram-share.ts تفعيل زر المشاركة	المرحلة 2 Share Engine
متوسطة تعديل هوك موجود إضافة مراقبة	use-live-prices.ts signals.tsx index.tsx	shared/hooks/use-signal-monitorts	بناء use-signal-monitorts تعديل useLivePrices تفعيل المراقبة في signals.tsx	المرحلة 3 Price Monitor
متوسطة تعديل صفحات موجودة	signals.tsx index.tsx analysis/\$id.tsx atoms.tsx	بدون ملفات جديدة	إضافة تحليل سريع ربط signal-tracking إضافة زر "تابع الإشارة"	المرحلة 4 Analysis UI
منخفضة	بدون	بدون	Build test اختبار يدوي لكل ميزة Git commit نشر على Vercel	المرحلة 5 اختبار ونشر

ما لن يتغير (حماية النظام)

قاعدة ذهبية: لا يتم حذف أو تعديل أي ملف موجود إلا إذا كان في قائمة "الملفات المتأثرة" اعلاه. المحرك التحليلي (/analysis/engine) لا يتغير. ال API endpoints في /server/api لا تتغير. ال 33 صفحة الموجودة لا تُحذف ولا تُستبدل. كل صفحة "COMING SOON" تبقى كما هي.

ملفات لن تُمس: كل ملفات محرك التحليل (/analysis/engine)، كل ملفات ال API (server/api/), كل ملفات ال Copilot، كل ملفات ال Discovery، كل ملفات ال Backtest، كل ملفات ال Wallet، كل ملفات ال Auth. هذه كلها تعمل ولا داعي لتعديلها.

صفحات لن تُمس: كل الصفحات ال 33 الموجودة تبقى كما هي. التعديلات ستكون اضافات فقط (ازرار جديدة، عناصر مرئية اضافية) وليس استبدالات.

مصفوف المخاطر والتخفيف

المخاطرة	الاحتمال	التخفيف
تعديل ملف موجود يكسر صفحة	متوسط	كل تعديل يكون اضافة (additive) وليس استبدال. لا نُعدل منطق موجود، نُضيف عليه فقط.
ال WebSocket يستهلك بطارية عالية	منخفض	الاشتراك محدود بازواج الاشارات النشطة فقط (5-10 ازواج عادة). BinanceWS يدعم Reconnect تلقائي.
Migration SQL يفشل في Supabase	متوسط	Migration بسيط (جدول واحد + فهرسين). لا يُعدل جداول موجودة. يمكن التراجع بسهولة.
تحديث types.ts يتعارض مع Supabase	منخفض	Types تُولّد من Supabase. بعد تنفيذ Migration، نُعيد توليد Types via CLI.
رابط المشاركة على X لا يعمل داخل تليجرام	متوسط	تليجرام يفتح الروابط في متصفح داخلي. X intent links تعمل في المتصفح. سنضيف fallback لـ "نسخ النص" ايضا.
Bundle size يزيد بسبب ملفات جديدة	منخفض	الملفات الجديدة صغيرة (200-400 سطر لكل ملف). لا نُضيف مكتبات جديدة. كل شيء يستخدم الموجود.